



TREINA RECIFE

DESENVOLVIMENTO GERENCIAL E PROFISSIONAL

Prof. Rodrigo M. M. Lyra

Curso de Lógica de Programação





Quem sou eu?

CEO da Empresa Rodrigo Lyra Consultoria Digital

Especialista em Gestão de Pessoas

Graduado em Gestão da Tecnologia da Informação

Graduado Comunicação – Habilitação em Relações Públicas

Professor de Lógica e Informática Básica na Treina Recife

Instrutor Cursos Executivos

- ***Desenvolvimento de Soluções Digitais e Banco de Dados.***
- ***Habilidades em gerir equipes, negociação, resolução de problemas e análise de dados.***



Prof. Rodrigo Lyra

 +55 81 9.8136-8793

 @rodrigo_lyra



<http://br.linkedin.com/rodrigolyrati>



rlyrastreeting@outlook.com



O que faz a Treina Recife ?



Formação de Programadores

Não está encontrando programadores para a sua empresa?
Nós formamos os desenvolvedores em um curso customizado para a sua empresa.
Temos um Programa que combina recrutamento com capacitação
Clique e saiba mais



Fábrica de Soluções

Está precisando desenvolver sistemas de pequeno porte, para automatizar as rotinas da sua empresa?
Temos uma Fábrica de Soluções em tecnologia sob medida, Inovação com IA, Web Service (integradores), Java, PHP, C#, Python.



Planejamento Estratégico

Já definiu os objetivos e construiu os indicadores de desempenho da sua empresa?
Prestamos Consultoria em Planejamento Estratégico, Diagnóstico, Objetivos, Mapa estratégico, Elaboração dos Planos de Ação, Metas e Indicadores: BSC ou OKR, Monitoramento.



Governança em TI

Governança em Tecnologia da Informação, Diagnóstico da infraestrutura de TI, suporte na instalação e configuração de solução ITSM e monitoramento da rede, servidores, máquinas virtuais e serviços em nuvem com ZABBIX e GRAFANA.



Prospecção de ERP

Vai comprar um novo ERP?
Nós podemos te ajudar na Elaboração de RFP, Prospecção no mercado, Análise de aderência e acompanhamento na implantação.



Gestão por Processos

Os processos da sua empresa estão mapeados?
Gestão por processos, Mapeamento, Redesenho, Documentação, Automação, Indicadores, Gap Analysis e Monitoramento.



Nossas Jornadas / Trilhas de Formação

JORNADA BACKEND



JORNADA FRONTEND



JORNADA DADOS





EMENTA

Parte 1: Conceitos e características da Linguagem JAVA

Parte 2: Conversões de Tipos; Comandos de entrada e saída.

Parte 3: Estruturas de decisão (If, If else, If else If, switch)

Parte 4: Estruturas de repetição (for, while, do while, for Each; Operadores Lógicos; Operadores aritméticos; Incremento e decremento



O que é uma linguagem de Programação?

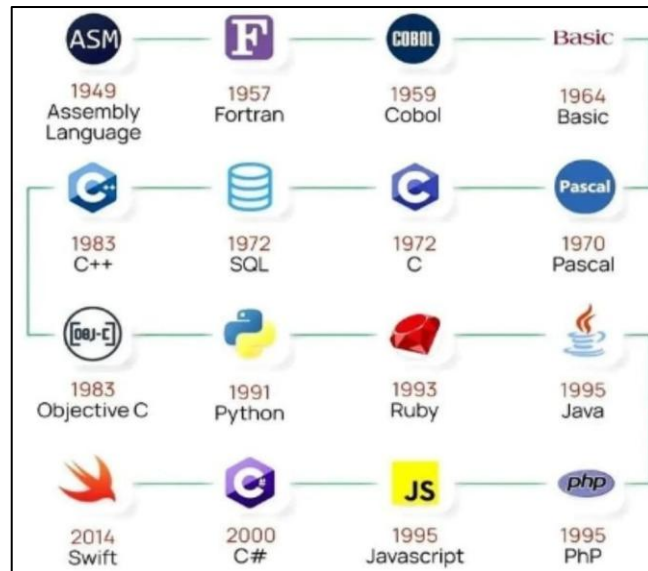
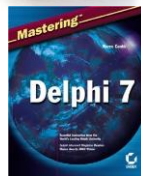
É uma notação formal para descrição de procedimentos que serão executados por um computador.



Exemplos:



python





O que é Lógica de Programação?

- A lógica de programação refere-se ao conjunto de princípios e técnicas usados para formular e implementar algoritmos que resolvem problemas computacionais.
- É a base que sustenta a construção de software e envolve o raciocínio e a organização das instruções que um computador deve seguir para realizar tarefas específicas.



O que é Programar?

Existem palavras comuns a qualquer pessoa que esteja aprendendo programação, como “algoritmos”, “diagramas de bloco”, “pseudocódigo” e “linguagens de programação”. Construir soluções computacionais baseadas em programação não se trata apenas de codificar.

Programar é uma forma de expressar, computacionalmente, uma determinada solução para um problema, de maneira que um computador seja capaz de executá-la. Já os **algoritmos** são uma das maneiras universais de expressar a resolução de um problema de maneira que qualquer pessoa seja capaz de resolver.

Os primeiros algoritmos do mundo são atribuídos à inglesa [Augusta Ada Byron](#) (conhecida como Condessa de Lovelace), em meados de 1840, embora seus trabalhos tenham sido publicados apenas em torno de 1950.

**** Algoritmos são uma sequência de passos computacionais que transformam uma entrada em uma saída. - Cormen (2009)***



O que é um Programa de Computador?

Conjunto de instruções descritas em uma **sequência lógica** com o objetivo de executar uma **determinada tarefa**, como por exemplo: realizar a soma de dois números, calcular o IMC de um paciente, calcular a folha de pagamento, reajustar valores de produtos, avançar a marcha de um câmbio automático.

Estrutura de um Programa

Instruções:

Corpo {

- Imperativas
- Condicionais
- Repetitivas

```
x = 5
```

```
input()
```

```
print('Olá!')
```

```
if (J < K): print(J)
```

```
if (J < K) :  
    print(J)  
else :  
    print(J-1)
```

```
while (M<4):  
    print(M)  
    M = M + 1
```

```
for m in range(0,4,1):  
    print (m)
```



O que é um Programa de Computador?

Em outras palavras, os algoritmos estão presentes no cotidiano mais do que se imagina. Trata-se de uma série de passos bem definidos e finitos para resolver um problema. A **entrada** é um conjunto de dados processados a partir de um conjunto de instruções que resultam em uma **saída**.

Os algoritmos podem ser expressos por meio de simples descrições narrativas, de fluxogramas, de pseudocódigos ou de linguagens de programação.

Descrições narrativas

Uma das principais vantagens das descrições narrativas é [utilizar a língua nativa](#) para estruturar os passos para a resolução do problema, porém ela pode fazer com que o algoritmo seja interpretado de maneiras diferentes, dificultando a execução da atividade ou a construção de programas.



O que é um Programa de Computador?

Imagine o quão difícil pode ser criar um algoritmo para fazer um sanduíche, por exemplo. Esse algoritmo descritivo poderia ser assim:

1	Pegue uma fatia de pão e coloque no prato.
2	Coloque uma fatia de queijo sobre a fatia de pão.
3	Coloque uma fatia de presunto sobre a fatia de queijo.
4	Coloque um pouco de alface sobre a fatia de presunto.
5	Coloque três fatias de tomate sobre o alface.
6	Coloque outra fatia de pão sobre os tomates.



O que é um Programa de Computador?

Se este algoritmo fosse analisado, era possível encontrar algumas ambiguidades, como: o que é “um pouco” de alface? Uma folha? Meia folha? Qual o tipo de prato a ser utilizado? Qual o tipo de queijo? As fatias de tomate são mais grossas ou mais finas?

Note que fazer um sanduíche é uma atividade de baixíssima complexidade, o que exigiria um nível de detalhamento muito menor nessa descrição, mas que ainda assim podem gerar dúvidas. Tarefas mais complexas podem exigir um esforço ainda maior para serem precisas.



O que é um Fluxograma?

Também conhecidos como **diagramas de bloco**, consistem em adotar formas geométricas e símbolos gráficos simples, que são previamente definidos, para guiar a resolução do problema. Uma das principais vantagens deste tipo de representação algorítmica é **facilitar a compreensão**.

Para isso, é necessário aprender os símbolos gráficos para representar o algoritmo apropriadamente. Vejamos alguns dos símbolos utilizados:



Representações gráficas de um Fluxograma:



SETA DE FLUXO

Indica o sentido do fluxo do algoritmo, conectando os símbolos existentes



ENTRADA DE DADOS

Indica entrada de dados



INICIADOR / TERMINADOR

Indica o início ou o fim do fluxograma



SAÍDA DE DADOS

Indica saída de dados



ATRIBUIÇÃO

Indica cálculos ou atribuição de valores



DESVIO CONDICIONAL

Indica uma tomada de decisão

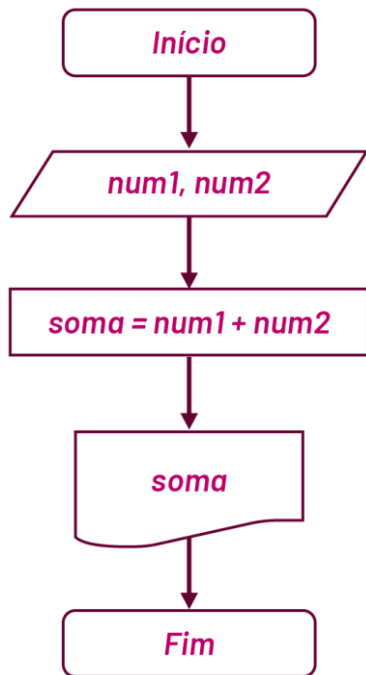


Representações gráficas de um Fluxograma:

Conhecendo os símbolos que fazem parte de um fluxograma, vamos considerar que você queira construir um algoritmo para representar a soma de dois números inteiros usando um fluxograma. No final da construção, você teria algo parecido com a imagem a seguir:



Fluxograma de um algoritmo da soma de dois números inteiros:



Os itens no campo de entrada de dados (num1, num2) são os valores a serem somados e o item “soma”, no campo de atribuição, recebe a soma desses dois valores, sendo esta a saída de dados, denominada também de “soma”. Todos esses itens recebem o nome de **variáveis**. É a partir das variáveis que podemos armazenar dados.



Exemplo de um Programa de Computador

Soma: Faça um programa para calcular e exibir a soma de dois números inteiros recebidos via teclado

<i>VisualAlg - Portugol</i>	<i>JAVA</i>	<i>Python</i>
<pre>algoritmo "SOMA" var X, Y, SOMA : INTEIRO inicio Leia(X) Leia(Y) SOMA := X + Y ESCREVA("a Soma foi:", Soma) Fimalgoritmo</pre>	<pre>public class SOMA { public static void main (String args[]) { int x, y, soma; x = system.in.read(); y = system.in.read(); soma = y + x; system.out.println("A Soma foi: "+ soma)); }</pre>	<pre>#Programa SOMA X = int(input()) Y = int(input()) SOMA = X + Y print ("A Soma foi:", SOMA)</pre>



O que é um Ambiente de Programação?

É um conjunto de **ferramentas de desenvolvimento** integradas que permite ao desenvolvedor construir os programas de computador;

Exemplo:



Visualg



Visual Studio Code



IntelliJ IDEA
JETBRAINS IDE



Configurações do Ambiente para a Disciplina

Utilizaremos o JDK, MAVEN e VS Code



Visual Studio Code
Aplicativo



Java™
Development Kit

Apache
Maven™



Configurações do Ambiente para a Disciplina



JDK (Java Development Kit)

O **JDK** O JDK é essencial para desenvolvedores Java, pois fornece tudo o que é necessário para criar, compilar e executar programas Java. O JDK é um kit de desenvolvimento para a linguagem de programação Java. Ele inclui:

- **JRE (Java Runtime Environment):** Um ambiente que fornece as bibliotecas e a máquina virtual Java (JVM) necessárias para executar programas Java.
- **Compilador Java (javac):** Um programa que compila código-fonte Java em bytecode, que a JVM pode executar.
- **Ferramentas de Desenvolvimento:** Inclui ferramentas como javadoc para gerar documentação e jar para empacotar arquivos de bytecode em arquivos JAR (Java ARchive).



Configurações do Ambiente para a Disciplina



<https://www.oracle.com/br/java/technologies/downloads/#java21>

Linux macOS Windows

Product/file description	File size
x64 Compressed Archive	185.91 MB
x64 Installer	164.28 MB
x64 MSI Installer	163.03 MB

JDK 23, Java 21, and earlier versions available now

JDK 23 is the latest release of the Java SE Platform.

JDK 21 is the latest Long-Term Support (LTS) release of the Java SE Platform.

Earlier JDK versions are available below.

JDK Development Kit 21.0.5 downloads

JDK 21 binaries are free to use in production and free to redistribute, at no cost, under the Oracle No-Fee Terms and Conditions (NFTC).

JDK 21 will receive updates under the NFTC, until September 2026, a year after the release of the next LTS. Subsequent JDK 21 updates will be licensed under the Java SE OTN License (OTN) and production use beyond the limited free grants of the OTN license will require a fee.

Linux macOS Windows

Product/file description	File size	Download
x64 Compressed Archive	185.91 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.zip (sha256)
x64 Installer	164.28 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.exe (sha256)
x64 MSI Installer	163.03 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.msi (sha256)



Configurações do Ambiente para a Disciplina



VS Code (Visual Studio Code)

VS Code é um editor de código-fonte leve e poderoso desenvolvido pela Microsoft. Ele é amplamente utilizado por desenvolvedores devido a suas características, incluindo:

- **Suporte a Múltiplas Linguagens;**
- **Extensibilidade;**
- **Debugging;**
- **Integração com Ferramentas de Desenvolvimento;**

VS Code é altamente configurável e pode ser ajustado para atender às necessidades específicas de desenvolvimento de diferentes linguagens e projetos.

Essas ferramentas são amplamente usadas no desenvolvimento de software moderno e oferecem suporte a diferentes aspectos do ciclo de vida do desenvolvimento, desde a codificação até a construção e gerenciamento de projetos.

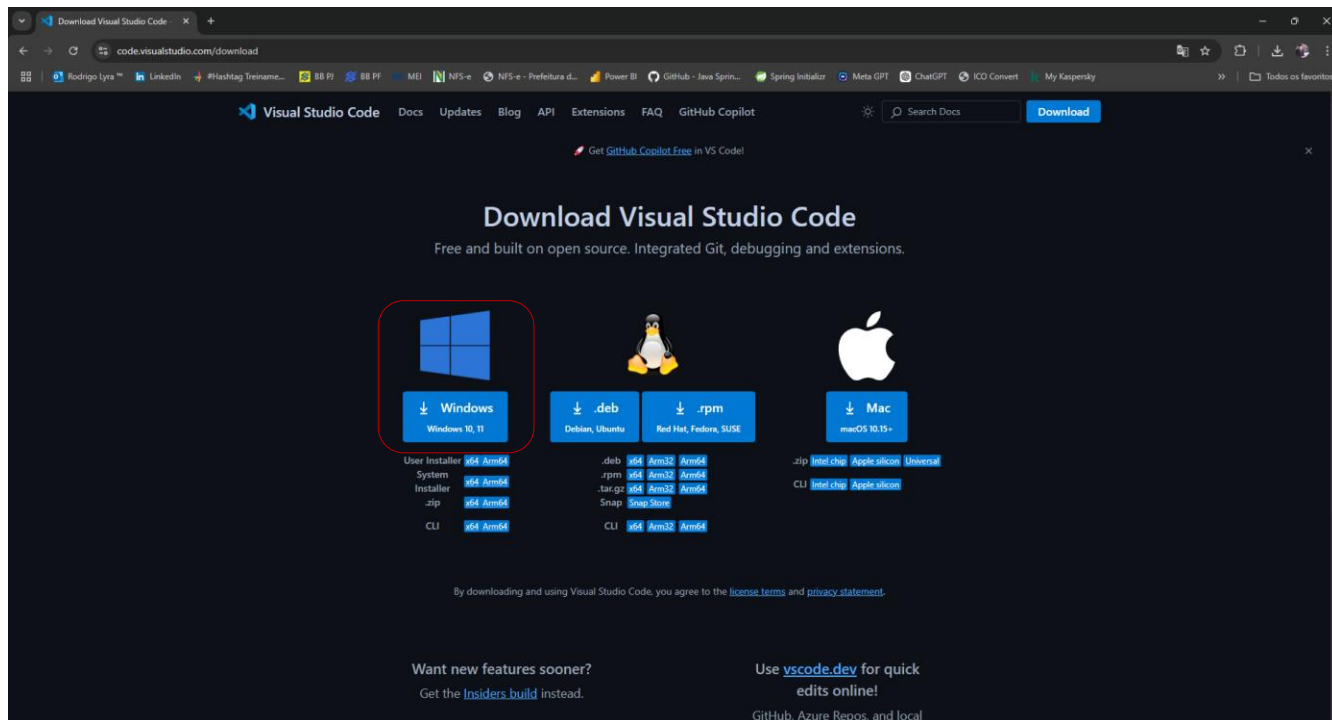


Configurações do Ambiente para a Disciplina



Visual Studio Code
Aplicativo

<https://code.visualstudio.com/download>





Configurações do Ambiente para a Disciplina



Maven

Maven é uma ferramenta de gerenciamento e automação de projetos para a linguagem Java, embora também possa ser usada com outros idiomas. Suas principais funções incluem:

- **Gerenciamento de Dependências:** Permite especificar bibliotecas e frameworks que seu projeto precisa, e o Maven baixa e gerencia essas dependências automaticamente.
- **Construção de Projetos:** Automatiza o processo de construção (compilação, empacotamento, teste) do seu projeto.
- **Estrutura de Projeto:** Segue uma convenção de estrutura de projeto padrão, o que facilita a organização e a manutenção dos projetos.
- **Plugins:** Oferece uma variedade de plugins que podem ser usados para tarefas como a execução de testes, análise de código e geração de relatórios.

O Maven usa um arquivo de configuração chamado pom.xml (Project Object Model) para definir as dependências, plugins e outras configurações do projeto.



Configurações do Ambiente para a Disciplina



<https://maven.apache.org/download.cgi>

Downloading Apache Maven 3.9.9

Apache Maven 3.9.9 is the latest release: it is the recommended version for all users.

System Requirements

Requirement	Details
Java Development Kit (JDK)	Maven 3.9+ requires JDK 8 or above to execute. It still allows you to build against 1.3 and other JDK versions by using toolchains .
Memory	No minimum requirement
Disk	Approximately 10MB is required for the Maven installation itself. In addition to that, disk space will be used for your local Maven repository. The size of your local repository will vary depending on usage but expect at least 500MB.
Operating System	No minimum requirement. Start up scripts are included as shell scripts (tested on many Unix flavors) and Windows batch files.

Files

Maven is distributed in several formats for your convenience. Simply pick a ready-made binary distribution archive and follow the [installation instructions](#). Use a source archive if you intend to build Maven yourself. In order to guard against corrupted downloads/installations, it is highly recommended to [verify the signature](#) of the release bundles against the public [KEYS](#) used by the Apache Maven developers.

	Link	Checksums	Signature
Binary tar.gz archive	apache-maven-3.9.9-bin.tar.gz	apache-maven-3.9.9-bin.tar.gz.sha512	apache-maven-3.9.9-bin.tar.gz.asc
Binary zip archive	apache-maven-3.9.9-bin.zip	apache-maven-3.9.9-bin.zip.sha512	apache-maven-3.9.9-bin.zip.asc
Source tar.gz archive	apache-maven-3.9.9-src.tar.gz	apache-maven-3.9.9-src.tar.gz.sha512	apache-maven-3.9.9-src.tar.gz.asc
Source zip archive	apache-maven-3.9.9-src.zip	apache-maven-3.9.9-src.zip.sha512	apache-maven-3.9.9-src.zip.asc

- 3.9.9 Release Notes and Release Reference Documentation
- latest source code from source repository
- Distributed under the Apache License, version 2.0
- other:

Link

Binary tar.gz archive	apache-maven-3.9.9-bin.tar.gz
Binary zip archive	apache-maven-3.9.9-bin.zip
Source tar.gz archive	apache-maven-3.9.9-src.tar.gz
Source zip archive	apache-maven-3.9.9-src.zip

- 3.9.9 Release Notes and Release Reference Documentation
- latest source code from source repository
- Distributed under the Apache License, version 2.0
- other:



Ter domínio sobre o assunto

Passos para elaboração de um Programa



Reservar as ferramentas necessárias



Elaborar um fluxo de execução



Escrever um algoritmo



**Traduzir o algoritmo para uma linguagem
de programação**



Testar o Programa



Tipos de dados

- **Primitivos**

Inteiros;

Números reais;

Booleanos;

Alfa-numéricos.

- **Não-Primitivos (Referências)**

Classes (String, Object, Livro, Conta, etc...);

Interfaces;

Arrays.



Tipos de dados

TIPO	TAMANHO (BITS)	VALOR MÍNIMO	VALOR MÁXIMO	SEM SINAL
boolean	1	false	true	X
char	16	0	$2^{16} - 1$	X
byte	8	-2^7	$2^7 - 1$	
short	16	-2^{15}	$2^{15} - 1$	
int	32	-2^{31}	$2^{31} - 1$	
long	64	-2^{53}	$2^{53} - 1$	
float	32			
double	64			



Tipos de dados

Tipo primitivo	Representação de constante literal	Exemplo
boolean	true ou false	boolean b = true ;
char	Um caractere entre aspas simples	char c = 'P';
byte	Número inteiro dentro da faixa do tipo byte	byte b = -111;
short	Número inteiro dentro da faixa do tipo short	short s = 2091;
int	Número inteiro dentro da faixa do tipo int	int i = 2098121;
long	Número inteiro, seguido da letra L, dentro da faixa do tipo long	long x = 290192011L;
float	Número decimal, seguido da letra F	float f = -291.22F
double	Número decimal	double d = 290.111



Conversão de Tipos de dados

A conversão dita explícita consiste em uma atribuição envolvendo tipos diferentes, mediada por um operador chamado CASTING, que significa “moldagem” em tradução livre. Tal operação é necessária para forçar a conversão de tipos mais “largos” em tipos mais “curtos”, evitando assim o erro de compilação que observamos quando tentamos realizar esta conversão de forma implícita e direta em uma atribuição.

Na conversão de um valor de tipo mais “largo”, para um tipo mais “curto”, pode haver perda de informação. Por exemplo, se um programador tenta converter um valor real com decimal diferente de zero em um inteiro, a parte decimal será descartada. Não há como armazenar internamente, nem mesmo representar casas decimais em uma variável de um tipo inteiro.



Conversão de Tipos de dados

O quadro a seguir mostra exemplos de quatro situações possíveis, respectivamente, em uma operação de CASTING.

- Conversão sem nenhuma perda.
- Conversão com perda da parte inteira.
- Conversão com perda da parte decimal.
- Conversão com perda das partes decimal e inteira.

Uma operação de CASTING consiste, normalmente, em uma linha de atribuição, em realizar uma conversão explícita de tipos no lado direito da atribuição, usando o operador de casting “(TIPO)”, conforme mostrado no quadro abaixo.

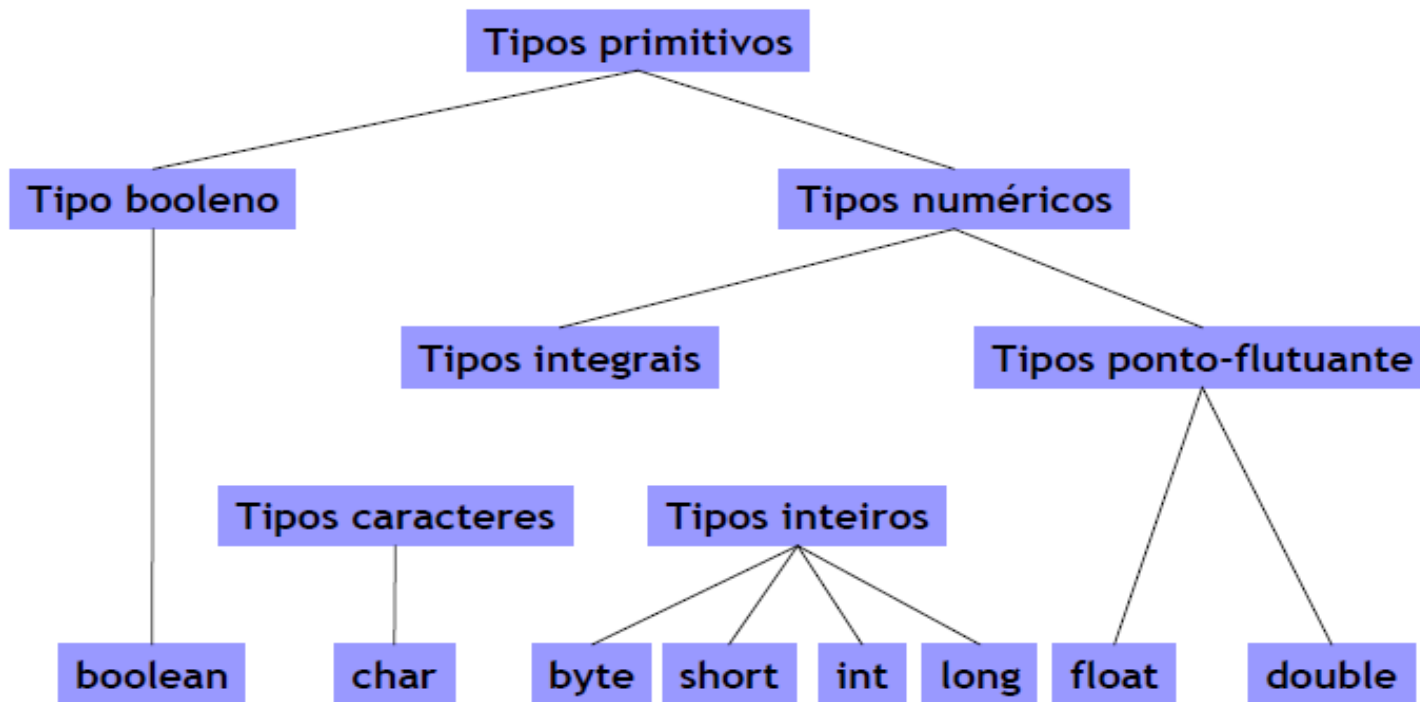


Conversão de Tipos de dados

Valores possíveis					
Inteiro	Primitivo	Menor	Maior	Valor Padrão	Tamanho
Inteiro	byte	-128	127	0	8 bits
	short	-32768	32767	0	16 bits
	int	-2.147.483.648	2.147.483.647	0	32 bits
	long	-9.223.372.036.854.770.000	9.223.372.036.854.770.000	0	64 bits
Ponto Flutuante	float	-1,4024E-37	3.40282347E + 38	0	32 bits
	double	-4,94E-307	1.79769313486232570E + 308	0	64 bits
Caractere	char	0	65535	\0	16 bits
Booleano	boolean	false	true	false	1 bits



Hierarquia de Tipos



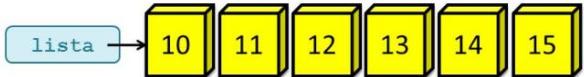


O que são Variáveis?

São espaços de memória reservados para guardar conteúdos, quando declaramos variáveis elas passam a ocupar espaços na memória RAM.

Exemplo: NOME, IDADE, SALARIO, NOTAS, etc...

Memória RAM

NOME 'José da Silva'	IDADE 29	SALARIO 1500.00
<pre>lista = [10, 11, 12, 13, 14, 15]</pre> Índices: 0 1 2 3 4 5 		<pre>#Lista de e-mails emails = ['r@gmail.com', 'r@yahoo.com'] emails[0]: r@gmail.com emails[1]: r@yahoo.com</pre>

Características

- SIMPLES
- COMPOSTAS
 - Unidimensionais (Listas)
 - Bidimensionais (array)



Declarando e Inicializando Variáveis

Para definir variáveis e constantes em JAVA dentro de um programa, basta declará-las conforme apresentado no quadro abaixo. No JAVA, o operador de atribuição é o “=” (igual). A definição de constantes exige o uso da palavra reservada **final**. O tipo pode ser qualquer tipo primitivo, ou qualquer tipo não primitivo (classe ou interface) disponível no projeto.

O nome é arbitrado pelo programador e a atribuição a um valor inicial, que pode ser uma constante (literal ou pré-definida), uma expressão ou outra variável. É opcional em se tratando de variáveis, mas, se presente, deve respeitar as restrições de conversão implícita e explícita da linguagem, que serão oportunamente apresentadas.

FONTE: Elaborada pela CESAR School



Palavras reservadas

abstract
boolean
break
byte
*byvalue**
case
catch
char
class
*const**

continue
default
do
double
else
extends
false
final
finally
float

for
*goto**
if
implements
import
instanceof
int
interface
long
native

new
null
package
private
protected
public
return
short
static
super

switch
synchronized
this
threadsafe
throw
transient
true
try
void
while

** Sem semântica na linguagem*



Convenções Java

- Maiúsculas e minúsculas fazem diferença
- Classes devem ser salvas em arquivos de mesmo nome com a extensão .java
- Pacotes deve corresponder a diretórios de mesmo nome.
- Nome de Classes são sempre escritos com a primeira letra maiúscula. Ex: Pessoa, Carro, CarroEsporte, ContaCorrente
- Nome de atributos e métodos sempre têm a primeira letra minúscula: nomePessoa, idade, dataNascimento, consultarPotencia.
- Nome de pacotes são escritos em minúsculo e geralmente não são nomes compostos. Ex: concessionaria, usuario.



Declarando e Inicializando Variáveis

Elemento da linguagem	Convenção para escrita de código	Exemplos
Classes e interfaces	Começam com letra MAIÚSCULA, são representadas com letras minúsculas e, caso o nome seja composto por várias palavras, as primeiras letras das palavras subsequentes são maiúsculas (padrão camel case). [Sem título]	<code>public class Funcionario</code> <code>public class PessoaFisica</code> <code>public interface Identificavel</code> <code>public interface CalculadorRenda</code>
Métodos, atributos e variáveis	Começam com letra MINÚSCULA, são representadas com letras minúsculas e, caso o nome seja composto por várias palavras, as primeiras letras das palavras subsequentes são maiúsculas (padrão camel case).	<code>void processar()</code> <code>double calcularRenda(int tipo)</code> <code>private double total</code> <code>private String nomeCompleto</code> <code>int quantidadePessoas</code> <code>Date hoje</code>
Constantes	São representadas com letras MAIÚSCULAS e, caso o nome seja composto por várias palavras, é colocado “_” entre uma palavra e outra.	<code>final int ZERO</code> <code>final double SALARIO_MINIMO</code>



Estrutura de um Programa Java

- Os programas começam com o método (função) `main()`:
- `public static void main(String[] args)`
args => passagem de argumentos para o programa Java.
- A execução da aplicação Java continua até que o método `main()` termine de executar todos os seus comandos (ou encontre o comando `System.exit(o)` antes.



Comentários

- Comentários são linhas que não serão executadas pelo programa, elas servem apenas para nos ajudar a entender melhor o que está sendo realizado.
- Poderemos comentar por exemplo, o porque estamos declarando uma variável, uma sua sub rotina ou até mesmo um trecho do código fonte.

Comentário de uma única linha

Utilizaremos duas barras //

// Isto é um comentário em uma linha

Exemplo:

//variável para cálculo de média

int media



Estrutura de um Programa Java

```
import java.util.Scanner; // Importa a classe Scanner

public class EntradaExemplo {
    public static void main(String[] args) {
        // Cria um objeto Scanner para capturar entradas do teclado
        Scanner scanner = new Scanner(System.in);

        // Exemplo 1: Captura uma string
        System.out.print("Digite seu nome: ");
        String nome = scanner.nextLine(); // Lê uma linha inteira
        System.out.println("Olá, " + nome + "!");

        // Exemplo 2: Captura um número inteiro
        System.out.print("Digite sua idade: ");
        int idade = scanner.nextInt(); // Lê um número inteiro
        System.out.println("Você tem " + idade + " anos.");

        // Exemplo 3: Captura um número decimal
        System.out.print("Digite sua altura (em metros): ");
        double altura = scanner.nextDouble(); // Lê um número decimal
        System.out.println("Sua altura é " + altura + " metros.");

        // Fecha o scanner
        scanner.close();
    }
}
```



Fluxo de um Algoritmo

Em lógica de programação, entrada, processamento e saída são conceitos fundamentais que descrevem como um programa de computador lida com dados para produzir resultados.

Esses conceitos ajudam a entender a estrutura básica de um algoritmo e são essenciais para a construção de qualquer programa.





Entrada

Definição: A entrada é qualquer dado ou informação que é fornecida ao programa para ser processada. Pode vir de várias fontes, como o teclado, arquivos, bancos de dados, ou até mesmo de outras aplicações ou sistemas.

Exemplos:

- Um usuário digitando seu nome em um formulário.
- Dados de um arquivo de texto que o programa lê.
- Informações de sensores em um dispositivo IoT (Internet das Coisas).

A entrada é fundamental porque fornece as informações necessárias para que o programa realize suas operações e produza resultados.



Processamento

Definição: O processamento é o conjunto de operações e cálculos que o programa realiza sobre a entrada para transformar dados em informações úteis ou resultados. Envolve a execução das instruções e algoritmos que manipulam os dados conforme as necessidades do programa.

Exemplos:

- Calcular a média de uma lista de números.
- Ordenar uma lista de nomes em ordem alfabética.
- Aplicar fórmulas matemáticas ou lógicas para obter um resultado específico.

O processamento pode envolver várias operações, como cálculos, comparações, iterações (loops), e manipulações de dados, e é o núcleo da lógica de programação.



Saída

Definição: A saída é o resultado do processamento, ou seja, a informação ou dados que o programa produz e exibe ao usuário ou que são enviados para outro sistema. A saída é a forma como o resultado do processamento é apresentado ou utilizado.

Exemplos:

- Mostrar uma mensagem ou um relatório na tela do usuário.
- Gravar resultados em um arquivo para posterior consulta.
- Enviar dados processados para outro sistema ou serviço.

A saída é o produto final que o programa oferece, permitindo que o usuário ou outros sistemas vejam ou utilizem os resultados do processamento.



Operadores Aritiméticos

Java	Operação	Exemplo	Resultado
+	Somar	$2 + 3$	5
-	Subtrair	$5 - 4$	1
*	Multiplicar	$3 * 4$	12
/	Dividir	$11 / 2$	5.5
%	Resto da divisão de valores	$11 \% 2$	1



Operadores Relacionais

Java	Operação	Exemplo
>	Maior que	7 > 3
>=	Maior ou igual	9 >= 9
<	Menor que	12 < 33
<=	Menor ou igual	16 <= 17
==	Igualdade	37 == 37
!=	Diferença	41 != 14

Obrigado!



+55 81 9.8136-8793



@rodrigo_lyra



<http://br.linkedin.com/rodrigolyra>



rlyrastreaming@outlook.com